REA: evaluación de incorporación

› Ingeniería inversa asistida por agentes para ELECTRONICA — guía para el HP EliteDesk 805 G6

Autor: Prof. CERO

Fecha: 4 de noviembre de 2026

Índice

1. **Resumen Ejecutivo** 2
2. **Qué es REA** 2
3. **Arquitectura** 3
4. **Cobertura** 4
5. **Encaje con ELECTRONICA** 5
6. **Valor para ELECTRONICA** 6
7. **Riesgos y Tensiones** 6
8. **Veredicto** 7
9. **Guía en la HP EliteDesk** 7
10. **Criterios de aceptación** 8

1 Resumen Ejecutivo

Se evaluó **REA** (*Reverse Engineer Anything*, `morluto/rea`, versión **6.4.0**, publicada el 11 de octubre de 2026) como herramienta de **ingeniería inversa asistida por agentes** que combina un **servidor MCP** y una **CLI** para reconstruir el funcionamiento de binarios, aplicaciones y comportamiento en tiempo de ejecución. Está pensada para que un asistente (OpenCode, Claude Code, Codex, Gemini CLI, Cursor, etc.) investigue un objetivo local, reciba hallazgos *con su evidencia* y los use para explicar, reproducir o reimplementar una funcionalidad.

El veredicto es **adopción condicional, y solo en el clon de ELECTRONICA sobre la PC HP EliteDesk 805 G6**. REA es una *capacidad nueva* —hoy el proyecto no analiza binarios, firmware ni ensamblados .NET— que encaja con el motor rotativo del asistente sin tocar `OPENROUTER_API_KEY`. Al mismo tiempo es una herramienta *exploratoria y no determinista* por naturaleza (descompilación + interpretación del LLM), así que **no puede formar parte de ningún flujo determinista** ni decidir lógica de negocio. Se adopta como *herramienta de consulta*, no como *flujo*.

Veredicto rápido: SÍ, adoptable, con condiciones

REA desbloquea análisis de **binarios nativos, firmware (binwalk/unblob), .NET, Go, WASM, bytecode EVM y aplicaciones JS/Electron** con evidencia auditable. Es local, MIT y complementa al RAG. **Condiciones:** (1) usarla solo como herramienta, nunca en las 3 capas como flujo; (2) proveedor **Ghidra headless** (libre) primero, y **Hopper / IDA diferidos** (comerciales); (3) vigilar el **disco de 256 GB** y el *payload* de esquemas MCP del contexto.

Recuerda

Vigilancia determinista. Adoptar REA como herramienta externa no viola las 3 capas: no introduce lógica de decisión en el chat ni un flujo nuevo. Sí lo violaría (a) resolver su *informe* en prosa de chat como si fuera una salida reproducible, o (b) crear un `flujo_*` que lo use sin su directiva y su script. Si la adopción deja de ser *herramienta* y pasa a *flujo* repetible, exige las tres capas: `directives/*.yaml` + orquestador + `execution/*.py`.

2 Qué es REA (y qué corrige la nota)

La nota de NotebookLM aportada por el usuario (`docs/AGENTE_IA/fuentes/rea-ingenieria-inversa-ia.md`) describe correctamente el espíritu de REA pero trae **imprecisiones de detalle** que conviene fijar antes de decidir nada. La tabla siguiente compara la nota con la fuente primaria verificada (repositorio y sitio oficial).

Ficha de la herramienta (verificada)

Repositorio	`morluto/rea` (MIT)
Sitio / Docs	`rea.tools`
Versión evaluada	**v6.4.0** (11 de octubre de 2026)
Paquete npm	`rea-agents` (CLI: `rea`)
Instalación	`npx rea-agents setup`
Runtime	**Node.js 22.19+/24.11+/26+** y npm
Formato	Servidor **MCP** + **CLI** (mismo motor)
Motores	Ghidra (libre), Hopper/IDA/JEB (comerciales)
Análisis	**Local**; cada resultado incluye evidencia y límites

Correcciones a la nota de NotebookLM

- El paquete/comando es `rea-agents` y `npx rea-agents setup`; la nota dice `rea-agent` (sin la "s").
- No son solo "Claude Code, Cursor, Codex y Gemini CLI": REA registra explícitamente **OpenCode** (`-client opencode`), Windsurf, Devin, Copilot CLI, Qwen Code, VS Code, Grok Build, OMP, Pi, Hermes y **DeepSeek Harness** (`-client dsh`, aún solo en `main`).
- El alcance es mayor: además de Ghidra/Hopper/.NET/Electron/Android, cubre **ELF, Go, WASM, bytecode EVM, sitios web, capturas HAR, firmware (binwalk/unblob)** y comportamiento de procesos.
- La nota omite los requisitos: **Node 22.19+** y que **Hopper, IDA y JEB son comerciales** (no vienen incluidos).

Concepto clave

La idea en una frase. "Mira una función de una app que te guste, entiende cómo funciona hasta el nivel del binario, y reconstruye una versión propia." REA no *copia* el software: devuelve *evidencia* (pseudocódigo, ensamblador, cadenas, referencias) para que el agente explique y reimplemente.

3 Cómo funciona por dentro

REA separa claramente *instalar la CLI* de *configurar los motores y el agente*. El agente llama a REA por **MCP**; la CLI ofrece los mismos flujos desde la terminal. Cada comando devuelve un registro de **Evidence**: el resultado junto con la evidencia que lo sustenta, sus supuestos y sus **límites** (lo que no pudo resolver).

Concepto clave

Motores de análisis (nativo). El análisis profundo de binarios usa uno de: **Ghidra** (libre, 12.1.x + JDK 21, Linux x64/arm64, macOS; Windows experimental), **Hopper** (comercial, demo; macOS y Linux con *Xvfb*), **IDA** (comercial; solo Windows verificado) o **JEB**. El análisis *estático* de JS/Electron, .NET, Go, WASM, ELF y paquetes **no necesita motor**: lee los ficheros sin ejecutarlos.

Límites declarados por el propio proyecto

- **Análisis local.** REA no sube la app: el objetivo se analiza en la máquina. El *proveedor de modelo* del agente sí aplica su propia política de datos (el informe que el agente redacte pasa por él).
- **Recursos propios.** Ghidra corre en modo `-readOnly` con proyecto efímero (`-deleteProject`); Hopper en Linux se ejecuta en un display privado.
- **Sesión única por objetivo.** Una sesión MCP mantiene un binario importado a la vez; cambiar de objetivo cierra el documento anterior.
- **Uso responsable.** Solo para software propio o autorizado (ver `DISCLAIMER.md`); las herramientas comerciales conservan sus licencias.

4 Qué objetivos puede analizar

REA cubre mucho más que "descompilar un ejecutable". La tabla resume los objetivos, lo que devuelve y el motor que exige (o si basta el análisis estático).

Objetivo	Qué devuelve	Requisito
Binarios nativos	Pseudocódigo, ensamblador, cadenas, símbolos, llamadas y referencias	Hopper / Ghidra / IDA
ELF (layout)	Secciones, segmentos, símbolos/relocalizaciones originales, mitigaciones	pwntools (Linux x64)
Binarios Go	Versión del compilador, módulos, *replacements* y *build settings*	**Nativo, sin Go**
WASM	Validación, secciones, *import/export* exactos y WAT decodificado	WABT
Bytecode EVM	Selectores de *dispatch*, *offsets*, argumentos y mutabilidad	Carrier local (Linux x64)
Crashes Linux	Hilos, registros, señales y candidatos de mapeo	pwntools (+ GDB)
JavaScript / Electron	Módulos, *imports*, *source maps*, rutas e IPC	**Solo Node**
Sitios web	Estructura de página, scripts y observaciones de red	Navegador Chrome
Capturas de red (HAR)	Peticiones, respuestas, *payloads* y su origen	HAR
Ensamblados .NET	Metadatos, instrucciones CIL y dependencias nativas	**Ninguno**
APK / Android	Manifiesto, clases y métodos descompilados	JADX + JDK
Firmware	Regiones, extracción y puenteo al análisis nativo	binwalk / unblob
Paquetes / recursos	Inventarios, *digests*, *plists*, anatomía de *bundles*	**Ninguno**

Objetivo	Qué devuelve	Requisito
Comportamiento de proceso	Salida de terminal, interacciones, E/S y comparación de corridas	PTY nativo (Linux)

5 ⚖ Encaje con las 3 capas y el gate determinista

Principio rector

El proyecto separa **orquestación probabilística** de **ejecución determinista**. REA es **puro análisis probabilístico**: la descompilación es reproducible, pero la *interpretación* la hace el LLM y varía. Por eso REA es una capacidad de **Capa 2 (herramienta)**, nunca de **Capa 3 (script determinista)**.

Aplicación del *gate* de determinismo (protocolo de fuentes externas, 2026-10-10) punto por punto:

Pregunta del gate	Evaluación para REA
¿Mete lógica de decisión en el prompt/-chat?	**Condicionado.** El *informe* de REA no debe decidir reglas de negocio. Se usa para *entender*, no para *decidir*.
¿Rompe reproducibilidad (mismo input $\neq$ output)?	**Sí, intrínsecamente.** Dos corridas pueden dar explicaciones distintas. Conclusión: **prohibido** dentro de flujos que exijan *mismo input* $\rightarrow$ *mismo output*.
¿Se salta I/O validadas, retry (máx 3) o la barrera anti-borrado?	**Sí, si se abusa del tiempo de ejecución.** Su captura en *runtime* ejecuta el objetivo con permisos del usuario. Solo sobre copias/objetivos de laboratorio; nunca sobre los entregables del repo.
¿Introduce estado persistente escribible por el modelo / inyección?	**A vigilar.** `setup` escribe configuración del agente (con *backups*, escritura atómica y verificación). No crea estado que el modelo escriba en el repo.
¿Duplica un mecanismo existente?	**Parcial.** El análisis de *websites* solapa con `execution/scrape_single_site.py`. Se descarta esa parte; el resto (binarios, firmware, .NET, HAR) es **capacidad nueva**.
¿Viola política de hardware/créditos?	**No por sí mismo.** Corre con Node local y no toca `OPENROUTER_API_KEY`. El gasto depende del motor del agente que interprete el resultado (rotativo, $0).

6 📈 Capacidades de valor por dominio

- **IoT / firmware (ESP32, PlatformIO).** Análisis de **firmware** (binwalk/unblob) y de binarios nativos: inspeccionar imágenes, extraer regiones y entender rutinas sin el código fuente. Encaja con la línea IoT del workspace.
- **Binarios de terceros.** Entender por qué una herramienta vendorizada se comporta de cierta forma, con evidencia (pseudocódigo + referencias) en vez de intuición.
- **.NET y Go.** Inspección estática **sin motor**: metadatos y CIL de ensamblados .NET; versión de compilador y módulos de binarios Go.
- **JS/Electron.** Mapear módulos, rutas e IPC de aplicaciones Electron o directorios JS (útil para el proyecto Node `Proyectos/cloudflare-agent`), con *source maps* y sin ejecutar la app.
- **Bytecode EVM.** Recuperar selectores y *mutabilidad* de contratos; útil para la línea blockchain (nota: Solana es Rust/SBF, **no** EVM).
- **Complementario al RAG.** El RAG responde "¿qué dice el proyecto?"; REA responde "¿qué hace este artefacto que no tenemos en texto?". No compiten.

</> Ejemplo

Ejemplo de flujo correcto. El agente pregunta "¿cómo exporta datos esta app Electron?" → REA devuelve módulos, IPC y origen de cada hallazgo → el agente redacta la explicación. El resultado es *asesor*, y su artefacto ("cómo funciona") no se convierte en una regla de negocio del repo.

7 Riesgos y tensiones

⚠ Tensiones a vigilar

- **Contexto del motor (MCP).** En `main`, REA expone del orden de **172 herramientas**; el perfil completo del catálogo ronda **496 KB** (≈120k *tokens* con el tokenizador o200k) y el perfil *compact* unos **290 KB** (≈68k *tokens*). Un motor con ventana pequeña puede "ahogarse" con los esquemas. Mitigación: fijar `REA_MCP_INPUT_SCHEMA_PROFILE` a `compact` y considerar solo los objetivos que el proyecto realmente usará.
- **Disco de 256 GB.** Ghidra + JDK **y** las dependencias de REA suman varios GB de instalación; el veredicto de hardware ya advierte que el 256 GB es la restricción real. Mitigación: instalar motores en el segundo slot de disco (M.2/2,5").
- **Motores comerciales.** Hopper, IDA y JEB **no se incluyen**; su demo tiene límites definidos por el fabricante. No usar rutas de activación no autorizadas.
- **Deriva `main` vs npm.** `dsh` y capacidades nuevas viven en `main` y no en la 6.4.0 de npm. Fijar versión y *no* seguir `latest` sin revisar el plan de `setup`.
- `setup` **edita configuración del agente.** Escribe entradas MCP en la config de OpenCode (con *backups*

y escritura atómica). `AGENTS.md` de este repo es la fuente de verdad: revisar el plan antes de aprobar cambios.

Regla de oro para el repo

Instalar y usar REA **solo con** `--dry-run` **revisado** primero. Ningún comando de REA debe apuntar al **propio repo como objetivo de escritura**: se analiza *software de terceros* o *firmware/binarios*, no los entregables versionados.

8 Veredicto

Adopción condicional (herramienta, no flujo)

Sí se adopta REA como herramienta de ingeniería inversa del clon de ELECTRONICA, **con estas condiciones no negociables:**

1. **Capa 2, jamás Capa 3.** No entra en flujos deterministas ni decide reglas; si un uso se vuelve repetible, se formaliza en las 3 capas.
2. **Ghidra primero.** Proveedor libre y *headless*; Hopper e IDA quedan **diferidos** hasta que haya caso de uso y presupuesto.
3. **Análisis estático primero.** JS/Electron, .NET, Go, WASM, ELF y paquetes no requieren motor ni ejecutan el objetivo: menor riesgo.
4. **Objetivos de terceros, sobre copias.** Nunca el repo versionado ni entregables; siempre software propio o autorizado.
5. **Perfil MCP compacto** y vigilancia del disco.

Temporalmente queda como **herramienta disponible en la HP EliteDesk**; su uso será *por demanda*. Reconvertirlo en flujo (p. ej. un `flujo_analizar_binario.py`) solo se justificará cuando exista una tarea repetible con entrada y salida verificables; entonces, y no antes, se escriben las tres capas.

9 Guía de implementación en la HP EliteDesk 805 G6

Este es el **equipo destino**: HP EliteDesk 805 G6 Mini (Ryzen 5 PRO 4650GE, Zen 2 6C/12T, **AVX2**, 16 GB RAM, SSD 256 GB). Sustituye al Celeron N4020 actual, que no tenía AVX2 ni RAM para cargas de análisis. Requisitos previos:

- **Node.js** del rango soportado. El workspace ya usa Node v22 vía `nvm` (el *plugin* `nvm-env.js` lo antepone al `PATH`); confirmar que sea ≥ 22.19 con `node -v`.
- **Ghidra 12.1.x + JDK 21** (libre) como motor nativo; o bien empezar solo con el análisis estático (no requiere

motor).

- Espacio en disco: reservar GBs para Ghidra, JDK y el caché de REA (idealmente el segundo disco).

</> Ejemplo

Secuencia prevista (revisar el plan antes de aprobar; *no* ejecutar aún):

```
# 1) Verificar runtime (nvm ya activo en las shells del workspace)
node -v                          # debe ser >= 22.19

# 2) Conectar Ghidra existente
export GHIDRA_INSTALL_DIR=/ruta/a/ghidra_12.1.4_PUBLIC
export JAVA_HOME=/ruta/a/jdk-21

# 3) Diagnostico del proveedor (solo lectura)
npx -y rea-agents@latest doctor --provider ghidra --json

# 4) Plan de setup para OpenCode SIN aplicar (revisar primero)
npx -y rea-agents@latest setup --client opencode --dry-run

# 5) Aplicar tras revisar el plan; reiniciar OpenCode despues
npx -y rea-agents@latest setup --client opencode
```

Recuerda

Fijar versión. Registrar con una versión concreta (`rea-agents@6.4.0`) en lugar de `@latest` para que la deriva de esquemas del proyecto no rompa la integración sin aviso. El perfil *compact* del catálogo MCP reduce el *payload*.

10 Criterios de aceptación y referencias

Criterios de aceptación del piloto (al llegar la HP):

1. `node -v` reporta dentro del rango soportado (22.19+/24.11+/26+).
2. `rea doctor -provider ghidra -json` devuelve el proveedor como listo (o se decide conscientemente arrancar en modo estático sin motor).
3. `setup -client opencode -dry-run` muestra el plan **sin** tocar `AGENTS.md` ni ficheros versionados; se aprueba a mano.
4. Un objetivo de laboratorio (p. ej. una app Electron o un binario propio) se analiza con resultado **con evidencia** y sin escribir en el repo.
5. Se documenta que ningún flujo determinista llama a REA.

Referencias primarias.

- Repositorio: https://github.com/morluto/rea
- Sitio y guías: https://rea.tools/ y https://rea.tools/get-started/
- Instalación y setup: https://github.com/morluto/rea/blob/main/docs/installation.md
- Nota fuente (secundaria, no citable): `docs/AGENTE_IA/fuentes/rea-ingenieria-inversa-ia.md`
- Análisis crítico (bitácora): `Sessions/2026-10-12_rea_evaluacion_incorporacion.md`